

PLUGINS

Create plugins

Create custom plugins to extend Claude Code with skills, agents, hooks, and MCP servers.

Plugins let you extend Claude Code with custom functionality that can be shared across projects and teams. This guide covers creating your own plugins with skills, agents, hooks, and MCP servers.

Looking to install existing plugins? See [Discover and install plugins](#). For complete technical specifications, see [Plugins reference](#).

When to use plugins vs standalone configuration

Claude Code supports two ways to add custom skills, agents, and hooks:

Approach	Skill names	Best for
Standalone (<code>.claude/</code> directory)	<code>/hello</code>	Personal workflows, project-specific customizations, quick experiments
Plugins (self-contained directories with skills, agents, hooks, or a <code>.claude-plugin/plugin.json</code> manifest)	<code>/plugin-name:hello</code>	Sharing with teammates, distributing to community, versioned releases, reusable across projects

Use standalone configuration when:

- You’re customizing Claude Code for a single project
- The configuration is personal and doesn’t need to be shared
- You’re experimenting with skills or hooks before packaging them
- You want short skill names like `/hello` or `/deploy`

Use plugins when:

- You want to share functionality with your team or community
- You need the same skills/agents across multiple projects

- You want version control and easy updates for your extensions
- You're distributing through a marketplace
- You're okay with namespaced skills like `/my-plugin:hello` (namespacing prevents conflicts between plugins)

💡 Start with standalone configuration in `.claude/` for quick iteration, then [convert to a plugin](#) when you're ready to share.

Quickstart

This quickstart walks you through creating a plugin with a custom skill. You'll create a manifest (the configuration file that defines your plugin), add a skill, and test it locally using the `--plugin-dir` flag.

Prerequisites

- Claude Code **installed and authenticated**

❗ If you don't see the `/plugin` command, update Claude Code to the latest version. See [Troubleshooting](#) for upgrade instructions.

Create your first plugin

1 Create the plugin directory

Every plugin lives in its own directory containing your skills, agents, or hooks, optionally alongside a `.claude-plugin/plugin.json` manifest. Create one now:

```
mkdir my-first-plugin
```

2 Create the plugin manifest

The manifest file at `.claude-plugin/plugin.json` defines your plugin's identity: its name, description, and version. Claude Code uses this metadata to display your plugin in the plugin

manager.

Create the `.claude-plugin` directory inside your plugin folder:

```
mkdir my-first-plugin/.claude-plugin
```

Then create `my-first-plugin/.claude-plugin/plugin.json` with this content:

```
my-first-plugin/.claude-plugin/plugin.json

{
  "name": "my-first-plugin",
  "description": "A greeting plugin to learn the basics",
  "version": "1.0.0",
  "author": {
    "name": "Your Name"
  }
}
```

Field	Purpose
name	Unique identifier and skill namespace. Skills are prefixed with this (e.g., <code>/my-first-plugin:hello</code>).
description	Shown in the plugin manager when browsing or installing plugins.
version	Optional. If set, users only receive updates when you bump this field. If omitted and your plugin is distributed via git, the commit SHA is used and every commit counts as a new version. See version management .
author	Optional. Helpful for attribution.

For additional fields like `homepage` , `repository` , and `license` , see the [full manifest schema](#).

3

Add a skill

Skills live in the `skills/` directory. Each skill is a folder containing a `SKILL.md` file. The folder name becomes the skill name, prefixed with the plugin's namespace (`hello/` in a plugin named `my-first-plugin` creates `/my-first-plugin:hello`).

Create a skill directory in your plugin folder:

```
mkdir -p my-first-plugin/skills/hello
```

Then create `my-first-plugin/skills/hello/SKILL.md` with this content:

```
my-first-plugin/skills/hello/SKILL.md
```

```
---  
description: Greet the user with a friendly message  
disable-model-invocation: true  
---
```

```
Greet the user warmly and ask how you can help them today.
```

4 Test your plugin

Run Claude Code with the `--plugin-dir` flag to load your plugin:

```
claude --plugin-dir ./my-first-plugin
```

Once Claude Code starts, try your new skill:

```
/my-first-plugin:hello
```

You'll see Claude respond with a greeting. Run `/help` to see your skill listed under the plugin namespace.

❗ **Why namespaces?** Plugin skills are always namespaced (like `/my-first-plugin:hello`) to prevent conflicts when multiple plugins have skills with the same name.

To change the namespace prefix, update the `name` field in `plugin.json`.

5 Add skill arguments

Make your skill dynamic by accepting user input. The `$ARGUMENTS` placeholder captures any text the user provides after the skill name.

Update your `SKILL.md` file:

```
my-first-plugin/skills/hello/SKILL.md
```

```
---
```

```
description: Greet the user with a personalized message
```

```
---
```

```
# Hello Skill
```

```
Greet the user named "$ARGUMENTS" warmly and ask how you can help them today. Make the greeting personal and encouraging.
```

Run `/reload-plugins` to pick up the changes, then try the skill with your name:

```
/my-first-plugin:hello Alex
```

Claude will greet you by name. For more on passing arguments to skills, see [Skills](#).

You've successfully created and tested a plugin with these key components:

- **Plugin manifest** (`.claude-plugin/plugin.json`): describes your plugin's metadata
- **Skills directory** (`skills/`): contains your custom skills
- **Skill arguments** (`$ARGUMENTS`): captures user input for dynamic behavior

💡 The `--plugin-dir` flag is useful for development and testing. When you're ready to share your plugin with others, see [Create and distribute a plugin marketplace](#).

Develop a plugin in your skills directory

Instead of passing `--plugin-dir` on every launch, you can keep a plugin in your skills directory and have Claude Code load it automatically. `claude plugin init` scaffolds one:

```
claude plugin init my-tool
```

This creates `~/.claude/skills/my-tool/` with a `.claude-plugin/plugin.json` manifest and a

starter `SKILL.md` . On the next session it loads as `my-tool@skills-dir` with no marketplace or install step.

For the auto-load rules, personal vs. project scope, the workspace-trust requirement, and how to update or remove one, see [Skills-directory plugins](#).

Plugin structure overview

You’ve created a plugin with a skill, but plugins can include much more: custom agents, hooks, MCP servers, LSP servers, and background monitors.

 **Common mistake:** Don’t put `commands/` , `agents/` , `skills/` , or `hooks/` inside the `.claude-plugin/` directory. Only `plugin.json` goes inside `.claude-plugin/` . All other directories must be at the plugin root level.

Directory	Location	Purpose
<code>.claude-plugin/</code>	Plugin root	Contains <code>plugin.json</code> manifest (optional if components use default locations)
<code>skills/</code>	Plugin root	Skills as <code><name>/SKILL.md</code> directories
<code>commands/</code>	Plugin root	Skills as flat Markdown files. Use <code>skills/</code> for new plugins
<code>agents/</code>	Plugin root	Custom agent definitions
<code>hooks/</code>	Plugin root	Event handlers in <code>hooks.json</code>
<code>.mcp.json</code>	Plugin root	MCP server configurations
<code>.lsp.json</code>	Plugin root	LSP server configurations for code intelligence
<code>monitors/</code>	Plugin root	Background monitor configurations in <code>monitors.json</code>
<code>bin/</code>	Plugin root	Executables added to the Bash tool’s <code>PATH</code> while the plugin is enabled
<code>settings.json</code>	Plugin root	Default <u>settings</u> applied when the plugin is enabled

A plugin that ships exactly one skill can place `SKILL.md` directly at the plugin root instead of creating a `skills/` directory. Claude Code loads it as a single skill and uses the frontmatter `name` field for the invocation name. Use the `skills/` layout for plugins that may grow to more than one skill.

❗ **Next steps:** Ready to add more features? Jump to [Develop more complex plugins](#) to add agents, hooks, MCP servers, and LSP servers. For complete technical specifications of all plugin components, see [Plugins reference](#).

Develop more complex plugins

Once you're comfortable with basic plugins, you can create more sophisticated extensions.

Add Skills to your plugin

Plugins can include **Agent Skills** to extend Claude's capabilities. Skills are model-invoked: Claude automatically uses them based on the task context.

Add a `skills/` directory at your plugin root with Skill folders containing `SKILL.md` files:

```
my-plugin/  
├── .claude-plugin/  
│   └── plugin.json  
└── skills/  
    ├── code-review/  
    │   └── SKILL.md
```

Each `SKILL.md` contains YAML frontmatter and instructions. Include a `description` so Claude knows when to use the skill:

```
---  
description: Reviews code for best practices and potential  
issues. Use when reviewing code, checking PRs, or analyzing code  
quality.  
---
```

When reviewing code, check for:

1. Code organization and structure
2. Error handling
3. Security concerns
4. Test coverage

After installing the plugin, run `/reload-plugins` to load the Skills. For complete Skill authoring

guidance including progressive disclosure and tool restrictions, see [Agent Skills](#).

Add LSP servers to your plugin

💡 For common languages like TypeScript, Python, and Rust, install the pre-built LSP plugins from the official marketplace. Create custom LSP plugins only when you need support for languages not already covered.

LSP (Language Server Protocol) plugins give Claude real-time code intelligence. If you need to support a language that doesn't have an official LSP plugin, you can create your own by adding an

`.lsp.json` file to your plugin:

`.lsp.json`

```
{
  "go": {
    "command": "gopls",
    "args": ["serve"],
    "extensionToLanguage": {
      ".go": "go"
    }
  }
}
```

Users installing your plugin must have the language server binary installed on their machine.

For complete LSP configuration options, see [LSP servers](#).

Add background monitors to your plugin

Background monitors let your plugin watch logs, files, or external status in the background and notify Claude as events arrive. Claude Code starts each monitor automatically when the plugin is active, so you don't need to instruct Claude to start the watch.

Add a `monitors/monitors.json` file at the plugin root with an array of monitor entries:

monitors/monitors.json

```
[
  {
    "name": "error-log",
    "command": "tail -F ./logs/error.log",
    "description": "Application error log"
  }
]
```

Each stdout line from `command` is delivered to Claude as a notification during the session. For the full schema, including the `when` trigger and variable substitution, see [Monitors](#).

Ship default settings with your plugin

Plugins can include a `settings.json` file at the plugin root to apply default configuration when the plugin is enabled. Currently, only the `agent` and `subagentStatusLine` keys are supported.

Setting `agent` activates one of the plugin's [custom agents](#) as the main thread, applying its system prompt, tool restrictions, and model. This lets a plugin change how Claude Code behaves by default when enabled.

settings.json

```
{
  "agent": "security-reviewer"
}
```

This example activates the `security-reviewer` agent defined in the plugin's `agents/` directory. Settings from `settings.json` take priority over `settings` declared in `plugin.json`. Unknown keys are silently ignored.

Organize complex plugins

For plugins with many components, organize your directory structure by functionality. For complete directory layouts and organization patterns, see [Plugin directory structure](#).

Test your plugins locally

Use the `--plugin-dir` flag to test plugins during development. This loads your plugin directly without requiring installation.

```
claude --plugin-dir ./my-plugin
```

The flag also accepts a `.zip` archive of the plugin directory, which requires Claude Code v2.1.128 or later.

```
claude --plugin-dir ./my-plugin.zip
```

When a `--plugin-dir` plugin has the same name as an installed marketplace plugin, the local copy takes precedence for that session. This lets you test changes to a plugin you already have installed without uninstalling it first. The exception is plugins that managed settings force-enable or force-disable: `--plugin-dir` cannot override those.

As you make changes to your plugin, run `/reload-plugins` to pick up the updates without restarting. This reloads plugins, skills, agents, hooks, plugin MCP servers, and plugin LSP servers. Test your plugin components:

- Try your skills with `/plugin-name:skill-name`
- Check that agents appear in `/agents`
- Verify hooks work as expected

 You can load multiple plugins at once by specifying the flag multiple times:

```
claude --plugin-dir ./plugin-one --plugin-dir ./plugin-two
```

To test a plugin that is already packaged as a `.zip` archive and hosted at a URL, such as a CI build artifact, use `--plugin-url` instead. Claude Code fetches the archive at startup and loads it for that session only. If the fetch fails or the archive is invalid, Claude Code reports a plugin load error and starts without it. The same **trust considerations** apply as for any plugin source: only point this flag at archives you control or trust.

To load multiple plugins, repeat the flag for each URL:

```
claude --plugin-url https://example.com/my-plugin.zip --plugin-url https://example.com/other.zip
```

Or pass space-separated URLs as one quoted argument:

```
claude --plugin-url "https://example.com/my-plugin.zip https://example.com/other.zip"
```

Debug plugin issues

If your plugin isn't working as expected:

1. **Check the structure:** Ensure your directories are at the plugin root, not inside `.claude-plugin/`
2. **Test components individually:** Check each skill, agent, and hook separately
3. **Use validation and debugging tools:** See [Debugging and development tools](#) for CLI commands and troubleshooting techniques

Share your plugins

When your plugin is ready to share:

1. **Add documentation:** Include a `README.md` with installation and usage instructions
2. **Choose a versioning strategy:** Decide whether to set an explicit `version` or rely on the git commit SHA. See [version management](#)
3. **Create or use a marketplace:** Distribute through [plugin marketplaces](#) for installation
4. **Test with others:** Have team members test the plugin before wider distribution

Once your plugin is in a marketplace, others can install it using the instructions in [Discover and install plugins](#). To keep a plugin internal to your team, host the marketplace in a [private repository](#).

Submit your plugin to the community marketplace

Anthropic maintains two public marketplaces for Claude Code plugins:

- `claude-plugins-official` : a curated set of plugins maintained by Anthropic. Registered automatically the first time you start Claude Code interactively. A non-interactive script that runs before that first launch must add it explicitly with `claude plugin marketplace add anthropics/claude-plugins-official` .
- `claude-community` : the public community marketplace where third-party submissions land after review. Users add it with `/plugin marketplace add anthropics/claude-plugins-community` and install from it as `@claude-community` .

To submit your plugin for community-marketplace review, use one of the in-app forms:

- **claude.ai:** claude.ai/admin-settings/directory/submissions/plugins/new
- **Console:** platform.claude.com/plugins/submit

The claude.ai form requires a Team or Enterprise organization and directory management access; organization Owners have this access by default. Individual authors who aren't part of a Team or Enterprise organization can use the Console form instead.

Run `claude plugin validate` locally before you submit. The review pipeline runs the same check on every submission, along with automated safety screening.

Approved plugins are pinned to a specific commit SHA in the `anthropics/claude-plugins-community` catalog, and CI bumps the pin automatically as you push new commits to your repository. The public catalog syncs nightly from the review pipeline, so there can be a delay between approval and your plugin appearing in `marketplace.json` . To check whether your plugin is installable yet, search for its name in the [community catalog](#).

The official marketplace, `claude-plugins-official` , is curated separately. Anthropic decides which plugins to include at its discretion. There is no application process, and the submission form does not add plugins to the official marketplace.

If Anthropic lists your plugin in the official marketplace, your CLI can prompt Claude Code users to install it. See [Recommend your plugin from your CLI](#).

❗ For complete technical specifications, debugging techniques, and distribution strategies, see [Plugins reference](#).

Convert existing configurations to plugins

If you already have skills or hooks in your `.claude/` directory, you can convert them into a plugin for

easier sharing and distribution.

Migration steps

1 Create the plugin structure

Create a new plugin directory:

```
mkdir -p my-plugin/.claude-plugin
```

Create the manifest file at `my-plugin/.claude-plugin/plugin.json` :

my-plugin/.claude-plugin/plugin.json

```
{
  "name": "my-plugin",
  "description": "Migrated from standalone configuration",
  "version": "1.0.0"
}
```

2 Copy your existing files

Copy your existing configurations to the plugin directory:

```
# Copy commands
cp -r .claude/commands my-plugin/

# Copy agents (if any)
cp -r .claude/agents my-plugin/

# Copy skills (if any)
cp -r .claude/skills my-plugin/
```

3 Migrate hooks

If you have hooks in your settings, create a hooks directory:

```
mkdir my-plugin/hooks
```

Create `my-plugin/hooks/hooks.json` with your hooks configuration. Copy the `hooks` object from your `.claude/settings.json` or `settings.local.json`, since the format is the same. The command receives hook input as JSON on stdin, so use `jq` to extract the file path:

my-plugin/hooks/hooks.json

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [{ "type": "command", "command": "jq -r
'.tool_input.file_path' | xargs npm run lint:fix" }]
      }
    ]
  }
}
```

4

Test your migrated plugin

Load your plugin to verify everything works:

```
claude --plugin-dir ./my-plugin
```

Test each component: run your commands, check agents appear in `/agents`, and verify hooks trigger correctly.

What changes when migrating

Standalone (`.claude/`)

Only available in one project

Files in `.claude/commands/`

Hooks in `settings.json`

Plugin

Can be shared via marketplaces

Files in `plugin-name/commands/`

Hooks in `hooks/hooks.json`

Standalone (`.claude/`)

Plugin

Must manually copy to share

Install with `/plugin install`

ⓘ After migrating, remove the original files from `.claude/` to avoid duplicates. Project and user `.claude/agents/` definitions override same-named plugin agents, so the plugin version only takes effect once the originals are removed.

Next steps

Now that you understand Claude Code's plugin system, here are suggested paths for different goals:

For plugin users

- **Discover and install plugins**: browse marketplaces and install plugins
- **Configure team marketplaces**: set up repository-level plugins for your team

For plugin developers

- **Create and distribute a marketplace**: package and share your plugins
- **Plugins reference**: complete technical specifications
- Dive deeper into specific plugin components:
 - **Skills**: skill development details
 - **Subagents**: agent configuration and capabilities
 - **Hooks**: event handling and automation
 - **MCP**: external tool integration